Chương 10

Shell Scripting

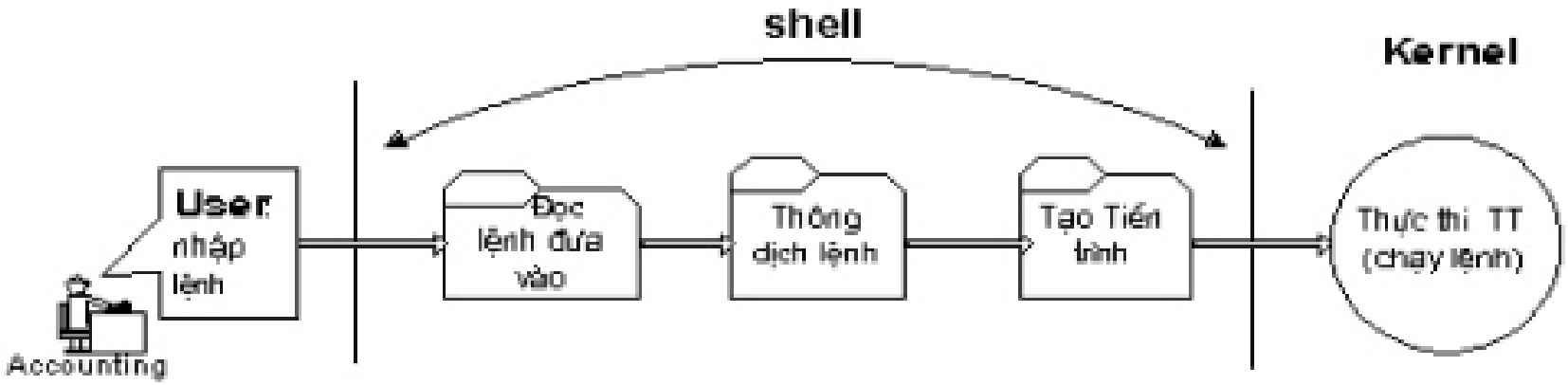
Số tiết: 6

Nội dung bài học

- Shell của Unix/Linux
- Sử dụng Shell như ngôn ngữ lập trình
- Cú pháp ngôn ngữ Shell

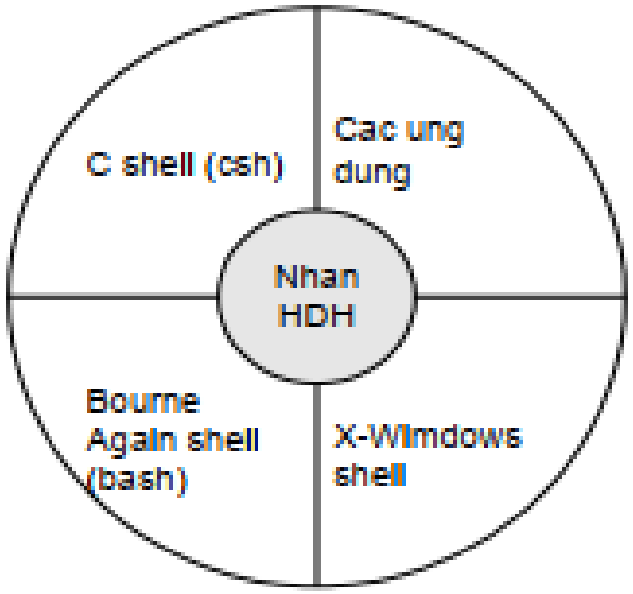
Shell của Unix/Linux

- Mọi thứ thực hiện trên Unix đều bởi tiến trình. Vậy tạo ra tiến trình như thế nào?
- Cách 1: Viết ra các chương trình biết tạo ra tiến trình (C/C++)
- Cách 2: Tận dụng phương tiện ***xử lý lệnh*** của hệ điều hành => Shell



Shell của Unix/Linux (tt)

- Tìm hiểu Shell thực tế là học một ngôn ngữ lập trình
- Unix/Linux có các loại Shell khác nhau và có thể chọn để dùng theo nhu cầu.



Shell của Unix/Linux (tt)

- Linux/ Unix tách biệt các ứng dụng, lệnh gọi các hàm chức năng của nhân thành những đơn thể nhỏ (tiến trình).
- Nhiều lệnh của Linux có thể kết hợp lại với nhau tạo nên chức năng tổng hợp rất mạnh mẽ.

Ví dụ: $ ls -al | more

- Kết hợp các lệnh với nhau chỉ bằng dòng lệnh không chưa đủ. Nếu muốn tổ hợp nhiều lệnh đồng thời với nhau và tùy vào từng điều kiện.. Phải dùng đến các lệnh rẽ nhánh: if, case.. Và lệnh lặp: for, while...

Shell của Unix/Linux (tt)

- Một số Shell trên Unix/Linux

a) **sh (Shell Bourne):** Shell nguyên thuỷ có mặt trên hầu hết các hệ thống Unix/Linux… Nó rất hữu dụng cho việc lập trình Shell nhưng nó không xử lý tương tác người dung như các Shell khác…

b) **bash (Bourne Again Shell):** Đây là phần mở rộng của sh, nó kế thừa những gì sh đã có và phát huy những gì sh chưa có. Đây là Shell được cài đặt mặc định trên các hệ thống Linux.

c) **Csh, tcsh và zsh:** Dòng shell sử dụng cấu trúc lệnh của C. Là shell thông dụng thứ 2 sau bash shell

Điều khiển shell bằng dòng lệnh

- Giả sử trên đĩa có nhiều file nguồn .c, cần hiển thị nội dung của tệp nguồn chứ chỗi main(). Thay vì dùng lệnh grep để tìm từng file sau đó quay lại lệnh more để hiển thị file. Ta có thể dùng shell như sau:

```
$ for file in *
do
        if grep -1 'main()' $file
        then
                more $file
        fi
done
```

Điều khiển shell bằng dòng lệnh (tt)

- Khi gõ một lệnh chưa hoàn chỉnh từ dấu nhắc của shell, shell sẽ chuyển dấu nhắc thành >
- Shell chờ nhập đầy đủ các lệnh trước khi thực hiện tiếp
- Bất tiện của phương pháp này là: Khó lấy lại khối lệnh trước đó để sửa đổi và thực thi một lần nữa.

Điều khiển shell bằng tập tin kịch bản (script file)

- Trước hết dùng lệnh

$cat > first.sh hay các trình soạn thảo như vi hay emacs (hoặc mc) để soạn nội dung tập tin first.sh như sau:

!/bin/sh

#first.sh

#Script này sẽ tìm trong thư mục hiện hành của choỗi

#mang nội dung main(), nội dung của file sẽ được hiển thị ra màn hình nếu tìm thấy

Điều khiển shell bằng tập tin kịch bản (script file) (tt)

```
for file in *
do
        if grep –l 'main()' $file
        then
                more $file
        fi
done
exit ()
```

Điều khiển shell bằng tập tin kịch bản (script file) (tt)

- Không như chú thích của C, một dòng chú thích (comment) của shell bắt đầu bằng #
- Tuy nhiên #!/bin/sh không phải là chú thích. Cạp ký tự #! Là chỉ thị yêu cầu shell hiện tại triệu gọi shell sh nằm trong thư mục bin. Shell sh sẽ chị trách nhiệm thông dịch các lệnh nằm trong tập tin script được tạo
- Chỉ thị #! Còn được dùng để gọi bất kỳ chương trình nào ta muốn chạy trước khi script tiếp theo được dịch
- Lệnh exit bảo đảm rằng sau khi thực thi sẽ trả về mã lỗi. Trả về 0 là thực thi thành công

Thực thi Script

- ***Cách 1: đơn giản nhất***

$/bin/sh <tên file.sh>

Ví dụ:

$bin/sh first.sh

- ***Cách 2: gọi first.sh ngay từ dòng lệnh***

$chmod +x <tên file.sh>

$<tên file.sh>

Ví dụ:

$chmod +x first.sh

$first.sh

Thực thi Script (tt)

- Có thể lệnh trên không thực hiện thành công và ta sẽ nhận thông báo lỗi ‘command not found’ (không tìm thấy lệnh)
- Lý do lỗi vì: biến môi trường PATH không chứa đường dẫn hay vị trí thư mục hiện hành.
- Cách khắc phục, thêm vào biến môi trường PATH chỉ định thư mục hiện hành như sau:

$PATH=$PATH:.

. Có thể chạy đơn giản:

$./<tên file.sh>

Thực thi Script (tt)

- Ngăn chặn script bị sửa bởi người dùng khác

#cp <tên file.sh> /usr/local/bin

#chown root /usr/local/bin/<tên file.sh>

#chgrp root /usr/local/bin/<tên file.sh>

#chmod u=rwx go=rx /usr/local/bin/<tên file.sh>

=> đoạn lệnh trên nhằm chuyển toàn quyền cho root. Nhóm khác chỉ được đọc và thực thi

Cú pháp ngôn ngữ SHELL

- Biến: kiểu chuỗi, kiểu số, tham số và biến môi trường
- Điều kiện: kiểm tra luận lý Boolean bằng shell
- Điều khiển chương trình: if, elif, for, while, until, case
- Hàm

Biến trong shell

- Trong linux shell thì có 2 kiểu biến:
- *Biến hệ thống (system variable): được tạo bởi Linux. Kiểu biến này thường được viết bằng ký tự in hoa.*
- *Biến do người dùng định nghĩa:*

Định nghĩa biến :

Cú pháp: tên biến=giá trị

Một số quy định về biến trong shell

(1) Tên bắt đầu bằng ký tự hoặc dấu gạch chân (_).

(2) Không được có khoảng trắng trước và sau dấu bằng khi gán giá trị cho biến

(3) Biến có phân biệt chữ hoa chữ thường

(4) Bạn có thể khai báo một biến có giá trị NULL như sau: var01= hoặc var01=“ ”

(5) Không dùng ?, * để đặt tên biến.

Một số quy định về biến trong shell (tt)

- Để truy xuất giá trị biến, dùng cú pháp sau: **$tên_biến**

  ví dụ :

```
n=10
echo $n
```

Lệnh trong Shell

- **Lệnh echo:** xuất nội dung ra màn hình

Ví dụ: vi bai1.sh

(có nội dung như sau)

#!/bin/sh

 # chuong trinh xuat ra man hinh mot chuoi

echo "chao cac ban"

Chạy chương trình:

#chmod a+x bai1.sh

#./bai1.sh

Lệnh trong Shell (tt)

- Lệnh read: đọc một số hay chuỗi từ bàn phím và gán cho tham_số
- read tham_số

Tính toán trong Shell

- Sử dụng expr:

cú pháp: expr op1 phép_toán op2

Ví dụ:

```
expr 1 + 3
expr 2 – 1
expr 10 / 2
expr 20 % 3
expr 10 * 3
echo `expr 6 \+ 3`
```

Tính toán trong Shell (tt)

- Sử dụng let

Ví dụ:

```
let "z=$z+3"
let "z += 3"
let "z=$m*$n"
```

Tính toán trong Shell (tt)

- Sử dụng $((...))

Ví dụ :

```
z=$((z+3))
z=$(($m*$n))
```

Chú ý khi dùng dấu nháy:

“ :Nháy kép bất cứ gì nằm trong dấy nháy kép được xem là những ký tự riêng biệt.

‘ :Nháy đơn những gì nằm trong dấu nháy đơn có ý nghĩa không đổi.

` :Nháy ngược thực thi lệnh

Tham số lệnh

- Tham số lệnh: Là tham số được truyền khi thực hiện script.

Ví dụ: ./bai2.sh phepcong 12 24

$1, $2, $3 … : vị trí và nội dung của các tham số trên dòng lệnh theo thứ tự từ trái sang phải.

$@, $*: danh sách tất cả các tham số trên dòng lệnh.

Điều kiện:

Nhận 1 trong 2 giá trị đúng hoặc sai

Sử dụng trong các câu lệnh điều khiển (if, while, ..)

Lệnh test hoặc []

Cấu trúc: test dieu_kien hoặc [dieu_kien]

Kiểm tra điều kiện tập tin

Ký hiệu	Mô tả
-d file	True nếu file là thư mục
-e fle	True nếu file tồn tại trên đĩa
-f file	True nếu file là tập tin thông thường
-g file	True nếu set-group-id được thiết lập trên file
-r file	True nếu file cho phép đọc
-s file	true nếu kích thước file khác 0
-u file	True nếu set-ser-id được áp đặt trên file
-w file	True nesu file cho phép ghi
-x file	True nếu file được phép thực thi

So sánh chuỗi

So sánh	Kết quả
String1 = string2	True nếu 2 chuỗi bằng nhau (chính xác từng ký tự)
String1 !=string2	True nếu 2 chuỗi không bằng nhau
-n string1	True nếu string1 không rỗng
-z string1	True nếu string1 rỗng (chuỗi null)

Các phép toán tử so sánh số học

-eq: bằng

-ge: lớn hơn hoặc bằng

-gt: lớn hơn

-le: nhỏ hơn hoặc bằng

-lt: nhỏ hơn

-ne: khác

Dấu bọc chuỗi (quoting)

- Dấu nháy kép được dùng trong trường hợp chuỗi của người dùng nhập vào có khoảng trắng. Tuy nhiên, bên trong dấu nháy kép, ký hiệu "**$**" vẫn có hiệu lực.

- Dấu nháy đơn có hiệu lực mạnh hơn. Nếu như tên biến có ký tự "**$**" đặt trong chuỗi có dấu nháy đơn → nó sẽ bị vô hiệu hóa. Nếu muốn hiển thị dấu "**$**" sử dụng dấu "\" trong chuỗi.

- Tóm lại, nếu muốn thay thế nội dung biến trong một chuỗi → dấu nháy kép. Còn nếu muốn hiển thị toàn bộ nội dung chuỗi → dấu nháy đơn.

Biến môi trường (Environment Variable)

- Khi trình shell khởi động đã tự động cung cấp một số biến được khai báo và gán giá trị mặc định → **BIẾN MÔI TRƯỜNG**

- Các biến môi trường này thường được viết hoa để phân biệt với các biến do người dùng định nghĩa. Một số biến môi trường chủ yếu sau:

Biến môi trường	Ý nghĩa
$HOME	Chứa nội dung của thư mục chủ (trong Home Directory)
$PATH	Chứa danh sách các đường dẫn (phân cách bằng dấu ":")
$PS1	Dấu nhắc hiển thị trên dòng lệnh ("$" → normal user)
$PS2	Dấu nhắc thứ cấp → user phải nhập thêm thông tin vào
$0	Chứa tên chương trình gọi trên dòng lệnh
$#	Số tham số truyền trên dòng lệnh
$IFS	Dấu phân cách các trường trong danh sách chuỗi

Biến tham số (Parameter Variable)

- Mục đích là để tiếp nhận tham số trên dòng lệnh cho việc xử lý
- Một số biến môi trường dưới đây:

Biến tham số	Ý nghĩa
$1, $2, S3, v.v...	Vị trí và nội dung của các tham số trên dòng lệnh theo thứ tự từ trái sang
$*	Danh sách tất cả các tham số trên dòng lệnh
$@	Danh sách các tham số được chuyển thành chuỗi

Biến tham số (Parameter Variable) (tt)

- Ví dụ sau sẽ cho thấy được sự khác nhau của hai biến $* và $@

```
#IFS="^"
#set foo bar bam
#echo "$@"
foo bar bam
#echo "$*"
foo^bar^bam
#unset IFS
#echo "$*"
foo bar bam
```

Biến tham số (Parameter Variable) (tt)

→ Lệnh **set** thiết lập 3 tham số dòng lệnh là **foo bar bam**. Những tham số này ảnh hưởng đến biến môi trường **$*** và **$@**

- Biến **$#** sẽ chứa số tham số dòng lệnh:

```
#echo "$#"
3
```

Điều kiện => Lệnh Test hoặc []

- Sử dụng lệnh **[]** hoặc **test** để kiểm tra điều kiện boolean (True or False)

- Lệnh **[]** trông đơn giản dễ hiểu, thường được dùng nhiều và rộng rãi hơn lệnh **test**

- Cách sử dụng hai lệnh trên là tương đương nhau

```
if test -f hello.c
then
        ...
fi
```

```
if [ -f hello.c ]
then
        ...
fi
```

→ Chú ý là phải đặt khoảng trắng giữa lệnh **[]** và biểu thức kiểm tra

Điều kiện => Lệnh Test hoặc [] (tt)

- Lệnh **test** có điều kiện trong đó cho phép kiểm tra một trong 3 kiểu sau:

  - So sánh chuỗi (Cần tham khảo thêm)

So sánh	Kết quả
string1 = string2	True nếu hai chuỗi bằng nhau (chính xác từng ký tự)
string1 != string2	True nếu hai chuỗi không bằng nhau
-n string1	True nếu string1 không rỗng
-z string1	True nếu string1 rỗng

  - So sánh toán học (Cần tham khảo thêm)

So sánh	Kết quả
exp1 –eq exp2	True nếu hai biểu thức bằng nhau
exp1 –ne exp2	True nếu hai biểu thức không bằng nhau
exp1 –ge exp2	True nếu exp1 lớn hơn hoặc bằng exp2
exp1 –le exp2	True nếu exp1 nhỏ hơn hoặc bằng exp2

Điều kiện => Lệnh Test hoặc [] (tt)

- So sánh điều kiện trên tập tin (Cần tham khảo thêm)

So sánh	Kết quả
-d file	True nếu file là thư mục
-e file	True nếu file tồn tại trên đĩa
-s file	True nếu kích thước file khác 0

Cấu trúc điều khiển -> lệnh if

- Chức năng: Kiểm tra điều kiện đúng hay sai để thực thi biểu thức thích hợp. Đây là lệnh được sử dụng nhiều nhất trong các chương trình (dù đó là chương trình lớn hay nhỏ)

- Cấu trúc:

```
if <điều kiện>
then
 <biểu thức lệnh>
else
 <biểu thức lệnh>
fi
```

Cấu trúc điều khiển -> lệnh elif

- Chức năng: Cũng tương tự như **if** → kiểm tra điều kiện đúng hay sai để thực thi biểu thức thích hợp. Lệnh này cho phép kiểm tra điều kiện lần thứ 2 bên trong **else**

Cấu trúc điều khiển -> các loại lệnh if

Dang 1

```
If command
Then
        Commands
Fi
```

Dang 2

```
If command
Then
        Commands
Else
        Commands
Fi
```

Dang 3

```
If command1
Then
        Command set 1
Elif command2
Then
        Command set 2
Elif command3
Then
```

Cấu trúc điều khiển -> So sánh dữ liệu kiểu số

- Phép so sánh trên dữ liệu kiểu số

So sánh	Mô tả
N1 eq *n2*	Đúng nếu n1 bằng n2.
N1 ge *n2*	Đúng nếu n1 lớn hơn hoặc bằng n2.
N1 gt *n2*	Đúng nếu n1 lớn hơn n2
N1 le *n2*	Đúng nếu n1 nhỏ hơn hoặc bằng n2
N1 lt *n2*	Đúng nếu n1 nhở hơn n2
N1 ne *n2*	Đúng nếu n1 không bằng n2

So sánh dữ liệu kiểu số -> ví dụ về lệnh if

- **Ví dụ if 1: so sánh 2 số nhập từ bàn phím**

```
Echo "Nhap so a:"
Read a
Echo "Nhap so b: "
Read b
If [ $a -lt $b ]
Then
        Echo "so $a nho hon so $b"
Elif [ $a -eq $b ]
Then
        Echo "so $a bang so $b."
Else
        Echo "so $a lon hon so $b.."
Fi
```

So sánh dữ liệu kiểu số -> ví dụ về lệnh if

- **Ví dụ if 2:**

```
#!/bin/bash
Val1=10
Val2=11
If [ $val1 -gt 5 ]
Then
        Echo "Gia tri $val1 lon hon 5"
Fi
If [ $val1 -eq $val2 ]
Then
        Echo "2 gia tri bang nhau"
Else
        Echo "hai gia tri khac nhau"
Fi
```

Phép so sánh trên dữ liệu kiểu chuỗi

So sánh	Mô tả
Str1 = str2	Đúng nếu chuỗi *str1 giống* chuỗi *str2*.
Str1 != str2	Đúng nếu chuỗi *str1 khác* chuỗi *str2*.
Str1 < str2	Đúng nếu chuỗi *str1 nhỏ hơn* chuỗi *str2*.
Str1 > str2	Đúng nếu chuỗi *str1 lớn hơn* chuỗi *str2*.
-n *str1*	Đúng nếu chuỗi *str1 có độ dài lớn hơn 0*.
-z *str1*	Đúng nếu chuỗi *str1 có độ dài bằng 0*.

Phép so sánh trên dữ liệu kiểu chuỗi => Ví dụ

- **Ví dụ if 3:**

```
#!/bin/bash
# so sanh bang
Read –p “nhập ten user:” testuser
If [ $USER != $testuser ]
Then
        Echo "User hien tai khong la $testuser"
Else
        Echo "Ban dang longin vơi $testuser"
Fi
```

Phép so sánh trên dữ liệu kiểu chuỗi => Ví dụ

- **Ví dụ if 4:**

```
#!/bin/bash
# kiem tra do dai chuoi
Val1=testing
Val2=''
If [ -n $val1 ]
Then
        Echo "The string '$val1' is not empty"
Else
        Echo "The string '$val1' is empty"
Fi
If [ -z $val2 ]
Then
        Echo "The string '$val2' is empty"
Else
        Echo "The string '$val2' is not empty"
Fi
If [ -z $val3 ]
Then
        Echo "The string '$val3' is empty"
Else
Echo "The string '$val3' is not empty"
Fi
$ ./ vidu6.sh
The string 'testing' is not empty
The string '' is empty
The string '' is empty
```

Phép toán kiểm tra file/Directory

Comparison	Description
-d *file*	Đúng nếu *file* là directory.
-e *file*	Đúng nếu *file* đang tồn tại.
-f *file*	Đúng nếu *file* là file.
-r *file*	Đúng nếu *file* là readable.
-s *file*	Đúng nếu *file* là không rỗng.
-w *file*	Đúng nếu *file* là writable.

-x *file*	Đúng nếu *file* là executable.
-O file	Đúng nếu *file* là sở hữu của user hiện hành.
-G *file*	Đúng nếu *file* là tồn tại và default group cũng là user hiện hành.
File1 -nt *file2*	Đúng nếu *mới hơn file2*.
File1 -ot *file2*	Đúng nếu *cũ hơn file2*.

Phép toán kiểm tra file/Directory => ví dụ

- **Ví dụ: Nhập tên file/directory cần kiểm tra**

```
#!/bin/bash
# kiểm tra file nhap vao la file hay thu mục

Read –p “ Nhap ten file, directory can kiem tra: “ file
If [ -e $file]
Then
        Echo "$file khong ton tai"
Elif [ -f $file]
Then
        Echo "$file la file!"
Elif [ -d $file]
Then
        Echo "$file la thu muc"
Fi
```

Các phép toán số học

Let được dùng để thực hiện các phép tính số học
Các toán tử số học : +, -, *, /, %
Các toán tử so sánh : >=, >, <, <=, = = , !=
Các toán tử logic : ! , && , | |

vidu8.sh - tính tong, hieu, tich, thuong tu 2 so nhap tu ban phim

```
Read –p " nhap 2 so a, b: " a b
Tong=` expr  $a  +  $b `
Echo "Tong của 2 số $a và $b là : $tong"
Hieu=` expr  $a  -  $b `
Echo "Hiệu của 2 số $a và $b là : $hieu"
Tich=` expr  $a  \*  $b `
Echo "Tích của 2 số $a và $b là : $tich"
Th=` expr  $a  /  $b `
Echo "Thương của 2 số $a và $b là : $th"
```

Cấu trúc điều khiển => lệnh for

- Chức năng: Để thực hiện việc lặp lại một số lần công việc với các giá trị xác định.

- Cấu trúc:

```
for <tên biến> in <các giá trị xác định>
do
 <biểu thức lệnh>
done
```

Ví dụ lệnh for

- **Ví dụ 1: Tạo ra các thư mục**

```
For test in ketoan kinhdoanh dao tao
Do
        Mkdir /$test
Done
```

Ví dụ lệnh for (tt)

- **Ví dụ 2:**

```
# kiem tra file, directory trong folder /etc
For file in /etc/*
Do
        If [ -d "$file" ]
        Then
                Echo "$file is a directory"
        Elif [ -f "$file" ]
        Then
                Echo "$file is a file"
        Fi
Done
```

Ví dụ lệnh for (tt)

- **Ví dụ 3:**

```
# vong lap sau thuc hien 10 lan
For ((  i=1;  i  <  10;  i++ ))
Do
        Echo "gia tri ke tiep la $i"
Done
```

Ví dụ lệnh for (tt)

- **Ví dụ 4:**

```
# 2 vong for long nhau

For (( i=1; i <= 10; i++ ))
Do
    Echo " gia tri ke tiep la $i"

    For (( j=1; j <= 10; j++ ))
    Do
            Echo " Inside loop: $j"
    Done
Done
```

Cấu trúc điều khiển => lệnh while

- Chức năng: Có chức năng như lệnh **for** nhưng nhằm đáp ứng được việc lặp trong một tập hợp lớn hoặc số lần lặp không biết trước.

- Cấu trúc:

```
while <điều kiện> do
 <biểu thức lệnh>
done
```

Ví dụ While

```
Var1=10
While [ $var1 -gt 0 ]
Do
        Echo $var1
        Var1=$[ $var1 - 1 ]    # tuong duong voi: var1=`expr $var1 - 1 `
Done
```

Cấu trúc điều khiển => lệnh until

- Chức năng: Có chức năng như lệnh **while** nhưng điều kiện bị đảo ngược lại. Vòng lặp sẽ bị dừng nếu điều kiện kiểm tra là đúng

- Cấu trúc:

```
until <điều kiện>
do
 <biểu thức lệnh>
done
```

Ví dụ until

```
Var1=100
Until [ $var1 -eq 0 ]
Do
        Echo $var1
        Var1=$[ $var1 - 25 ]
Done
```

Ví dụ kết hợp for và while

```
Var1=5
While [ $var1 -ge 0 ]
Do
        Echo "Outer loop: $var1"
        For (( var2 = 1; $var2 < 3; var2++ ))
        Do
                Var3=$[ $var1 * $var2 ]
                Echo " Inner loop: $var1 * $var2 = $var3"
        Done
        Var1=$[ $var1 - 1 ]
Done
```

Ví dụ kết hợp for và while (tt)

```
Var1=3
Until [ $var1 -eq 0 ]
Do
        Echo "Outer loop: $var1"
        Var2=1
        While [ $var2 -lt 5 ]
        Do
                Var3=`echo "scale=4; $var1 / $var2" | bc`
                Echo " Inner loop: $var1 / $var2 = $var3"
                Var2=$[ $var2 + 1 ]
        Done
        Var1=$[ $var1 - 1 ]
Done
```

Cấu trúc điều khiển => lệnh case

- Chức năng: Có chức năng là cho phép ta so khớp nội dung của biến với một mẫu chuỗi (pattern) nào đó. Khi một mẫu được so khớp thì <biểu thức lệnh> tương ứng sẽ được thực hiện.

- Cấu trúc:

```
case <tên biến> in
 mẫu chuỗi [ | mẫu chuỗi] ...) <biểu thức lệnh>;;
 mẫu chuỗi [ | mẫu chuỗi] ...) <biểu thức lệnh>;;
 ...
esac
```

Danh sách thực thi lệnh

- Shell cung cấp cho ta cú pháp danh sách AND và OR để có thể kết nối các lệnh lại với nhau theo thứ tự kiểm tra trước khi ra một quyết định nào đó.
- **Danh sách AND**

- Chức năng: Cho phép thực thi một chuỗi lệnh kề nhau. Lệnh sau chỉ thực hiện khi lệnh trước đã thực thi và trả về mã lỗi thành công

- Cấu trúc:

```
<biểu thức lệnh 1> && <biểu thức lệnh 2> && ...
```

Danh sách thực thi lệnh (tt)

- **Danh sách OR**

  - Chức năng: Cũng như lệnh **AND** là thực thi một dãy các lệnh NHƯNG nếu có một lệnh trả về là **TRUE** thì việc thực thi dãy lệnh sẽ dừng lại

  - Cấu trúc:

```
<biểu thức lệnh 1> || <biểu thức lệnh 2> || ...
```

  - Kết quả cuối cùng của danh sách **OR** chỉ đúng (TRUE) khi có một trong các <biểu thức lệnh> trả về TRUE

  - Khác với "&&" là gọi lệnh tiếp theo khi các lệnh trước đó là TRUE còn với "||" thì gọi lệnh tiếp theo trong chuỗi lệnh khi lệnh trước đó là FALSE

Hàm (function)

- Shell cho phép ta tự tạo lập các hàm hay thủ tục để triệu gọi bên trong Script

- Ta có thể gọi các script con khác bên trong script chính ☞ tuy nhiên việc này thường làm tiêu tốn tài nguyên và không hiệu quả bằng triệu gọi hàm

- Cấu trúc:

```
tên hàm() {
 <biểu thức lệnh1>
 <biểu thức lệnh2>
 . . .
 <biểu thức lệnhn>
}
```

Hàm (function) (tt) => Ví dụ

Ví dụ tạo hàm trong file shell script:

```
#!/bin/bash
hello()
{echo "Executing function hello"
}
echo "Script has started now"
hello
echo "Script will end "
```

Thực hiện chạy chương trình:

```
$ chmod +x function_01.sh
$ ./function_01.sh
```

Kết quả:

```
Script has started now
Executing function hello
Script will end
```

Hàm (function) (tt)

- Biến cục bộ và biến toàn cục

  - Khai báo biến cục bộ (chỉ có hiệu lực bên trong hàm) ☞ dùng từ khoá **local**. Do vậy, nếu không có từ khóa trên thì biến chỉ được hiểu là toàn cục (global)

  - Phạm vi lưu trữ của biến toàn cục không còn hiệu lực khi hàm kết thúc

  - Biến toàn cục được nhìn thấy và có thể thay đổi bởi tất cả các hàm trong cùng script.

Hàm (function) (tt)

- **Hàm và cách truyền tham số**

- Shell không cung cấp chức năng khai báo tham số cho hàm.
- Việc truyền tham số cho hàm tương tự truyền tham số trên dòng lệnh
- Ví dụ: Truyền tham số cho foo() ☞ foo “tham số 1”, “tham số 2”, ...

Các lệnh nội tại của Shell

- **Lệnh Break:** Thoát khỏi vòng lặp **for, while** hoặc **until** bất kể điều kiện thoát của các lệnh này có diễn ra hay không.

Ví dụ:

```
For var1 in 1 2 3 4 5 6 7 8 9 10
Do
        If [ $var1 -eq 5 ]
        Then
                Break
        Fi
        Echo "Gia tri tiep theo: $var1"
Done
Echo "Vong lap for hoan thanh"
```

Các lệnh nội tại của Shell (tt)

- **Lệnh continue:** Thường được dùng bên trong vòng lặp, lệnh này yêu cầu vòng lặp quay ngược lại thực hiện bước lặp kế tiếp mà không cần thực thi các khối lệnh còn lại.

Ví dụ:

```
# su dung lenh continue voi for
For (( var1 = 1; var1 < 15; var1++ ))
Do
        If [ $var1 -gt 5 ] && [ $var1 -lt 10 ]
        Then
                Continue
        Fi
        Echo "Iteration number: $var1"
Done
```

Các lệnh nội tại của Shell (tt)

- **Lệnh :**

  - Chức năng: Là một lệnh rỗng (NULL). Đôi lúc lệnh này được dùng với ý nghĩa logic là TRUE. Việc dùng lệnh "**:**" sẽ thực thi nhanh hơn việc so sánh true. Như "**while :**" sẽ nhanh hơn "**while true**"

  - Ghi chú: Một số shell phiên bản cũ sử dụng lệnh "**:**" như là một chú thích lệnh ~ như "#"

Các lệnh nội tại của Shell (tt)

- **Lệnh . (thực thi)**

- Chức năng: Dùng để thực thi một script bên trong shell hiện hành. Đồng thời khi thực thi chính lệnh "." sẽ giữ nguyên những thay đổi về môi trường mà các biến tác động lên (do khi thực thi một script, shell sẽ lưu lại toàn bộ biến môi trường và tạo ra môi trường mới - sub shell - để script có thể hoạt động và các thông số của biến môi trường sẽ được khôi phục lại khi script chấm dứt - bằng lệnh exit())

Các lệnh nội tại của Shell (tt)

- **Lệnh eval**

  - Chức năng: Cho phép thực hiện một lệnh động phụ thuộc vào biến

  - **Ví dụ 1:**

```
$ foo=10
$ x=foo
$ y='$'$x
$ echo $y
```

⇨ Kết quả in ra là **$foo**

Các lệnh nội tại của Shell (tt) => lệnh eval

- Ví dụ 2:

```
$ foo=10
$ x=foo
$ eval y=‘$’$x
$ echo $y
```

⇨Kết quả in ra là 10

Các lệnh nội tại của Shell (tt)=> Lệnh exec

- Chức năng: Dùng để gọi một lệnh bên ngoài khác. Thông thường lệnh **exec** sẽ gọi một shell phụ khác với shell mà script đang thực thi.

- Mặc định thì **exec** sẽ triệu gọi lệnh **exit** khi kết thúc lệnh ➔ Do đó nếu ta gọi lệnh **exec** ngay từ dòng lệnh thì sau khi thực thi lệnh xong (do gọi tiếp lệnh **exit**) ta sẽ bị thoát ra khỏi shell hiện hành và quay trở về màn hình đăng nhập.

Các lệnh nội tại của Shell (tt)=> Lệnh exit n

- Chức năng: Dùng để thoát ra khỏi shell đang gọi và trả về mã lỗi n

- Tương tự như trên nếu như ta gọi **exit** ngay từ dòng lệnh thì ta sẽ thoát ra khỏi shell hiện hành và quay về màn hình đăng nhập.

- Mã lỗi: tham khảo thêm trong giáo trình.

Các lệnh nội tại của Shell (tt)=> Lệnh export

- Chức năng: Do khi thực thi một shell thì các biến môi trường đều được lưu lại. Như vậy, khi khai báo và sử dụng các biến trong một script thì các biến này chỉ có giá trị của shell triệu gọi script đó.
⇨ Do vậy, lệnh **export** được đề cập ở đây cho phép các biến có thể thấy được tất cả các script trong shell phụ hay các script được triệu gọi từ shell khác.

- Lệnh **export** có chức năng như khai báo biến toàn cục

Các lệnh nội tại của Shell (tt)=> Lệnh expr

- Chức năng: Ước lượng giá trị đối số truyền cho nó như là một biểu thức và thường được dùng trong việc tính toán kết quả toán học đổi từ chuỗi sang số. Chú ý: Biểu thức có lệnh **expr** đặt trong cặp dấu “` `”

Các lệnh nội tại của Shell (tt)=> Lệnh printf

- Chức năng: Tương tự như lệnh **printf** của thư viện C

- Danh sách các ký tự đặc biệt dùng chung với dấu "\", gọi là chuỗi thoát

Chuỗi thoát	Ý nghĩa
\\	Cho phép hiển thị dấu "\" trong chuỗi
\a	Phát tiếng Beep
\b	Ký tự xóa BackSpace
\f	Đẩy dòng
\r	Về đầu dòng
\t	Canh TAB ngang
\v	Canh TAB dọc
\ooo	Ký tự đơn với mã ký tự là ooo
\n	Xuống dòng mới

Các lệnh nội tại của Shell (tt)=> Lệnh printf

- Định dạnh số và chuỗi bằng ký tự %

Ký tự định dạng	Ý nghĩa
d	Số nguyên
c	Ký tự
s	Chuỗi
%	Hiển thị ký hiệu %

Các lệnh nội tại của Shell (tt)=> Lệnh return

- Chức năng: Trả về giá trị của hàm
- Nếu lệnh không có tham số thì sẽ trả về mã lỗi của lệnh vừa thực hiện

Các lệnh nội tại của Shell (tt)=> Lệnh set

- Chức năng: Dùng để thiết lập giá trị cho các biến môi trường như **$1**, **$2**, **$3**,... Ngoài ra, lệnh này còn có chức năng loại bỏ những khoảng trắng không cân thiết và đặt nội dung của chuỗi truyền cho nó vào các biến tham số

```
#set    This    is    parameter
#echo $1
This
#echo $3
parameter
```

Các lệnh nội tại của Shell (tt)=> Lệnh shift

- Chức năng: Di chuyển nội dung các tham số môi trường **$1**, **$2**, **$3**, v.v... xuống một vị trí. (Do ta chỉ có tối đa 9 tham số $1..$9)

- Nếu gọi tham số $10 thì sẽ được hiểu là $1 và "0"

Các lệnh nội tại của Shell (tt)=> Lệnh trap

- Chức năng: Dùng để bẫy một tín hiệu (signal) do hệ thống gửi đến Shell trong quá trình thực thi script

- Tín hiệu ở đây thông thường là một thông điệp của hệ thống gởi đến chương trình yêu cầu hay thông báo một công việc nào đó mà hệ thống sẽ thực hiện. Ví dụ: Ngắt INT thường được gởi khi nhấn CTRL+C

Tín hiệu	Ý nghĩa
HUP(1)	Hang-up, nhận khi người dùng logout
INT(2)	Interrupt, ngắt khi nhấn CTRL+C
QUIT(3)	Quit, thoát khi nhấn CTRL+\

Các lệnh nội tại của Shell (tt)=> Lệnh unset

- Chức năng: Loại bỏ biến khỏi môi trường Shell
- Lệnh **unset** rất ít được sử dụng
- Ví dụ:

```
#!/bin/sh
foo="Hello World"
echo $foo
unset foo
echo $foo
```

- Kết quả chương trình sẽ in ra chuỗi rỗng (do lúc này biến foo không tồn tại nữa)

Dò lỗi (Debug) của Script

- Do script là lệnh văn bản được shell thông dịch nên việc dò lỗi không khó như các chương trình biên dịch nhị phân

- Quá trình dò lỗi thì shell sẽ in ra số thứ tự của dòng gây lỗi. Ta cũng có thể thêm vào lệnh **echo** để in ra nội dung của các biến có khả năng gây lỗi cho chương trình

- Ta có thể dùng **set** để đặt một số tùy chọn cho shell hoặc đặt thêm tham số khi gọi shell thực thi script

Dò lỗi (Debug) của Script (tt)

Tham số dòng lệnh cho Shell	Đặt tùy chọn bằng *set*	Ý nghĩa
sh -n <scripts>	set –o noexec set –n	Chỉ kiểm tra cú pháp, không thực thi lệnh
sh -v <scripts>	set –o verbose set –v	Hiển thị lệnh trước khi thực hiện
sh -x	set –o xtrace set –x	Hiển thị lệnh sau khi thực hiện
sh -u	set –o nounset set –u	Hiển thị thông báo lỗi khi một biến sử dụng nhưng chưa được định nghĩa

Viết ứng dụng với Shell

```
# tao menu
Function diskspace {
        Clear
        Df -k
}

Function whoseon {
        Clear
        Who
}

Function memusage {
        Clear
        Cat /proc/meminfo
}

Function menu {
        Clear
        Echo
        Echo -e "\t\t\tsys Admin Menu\n"
        Echo -e "\t1. Display disk space"
        Echo -e "\t2. Display logged on users"
        Echo -e "\t3. Display memory usage"
        Echo -e "\t0. Exit program\n\n"
        Echo -en "\t\tenter option: "
        Read -n 1 option
}
```

```
# su dung option -e, cho phep echo su dung ki tu
# dieu khien dac biet, vd \t kq: tab
While [ 1 ]
Do
        Menu
        Case $option in
        0)
                Break ;;
        1)
                Diskspace ;;
        2)
                Whoseon ;;
        3)
                Memusage ;;
        *)
                Clear
                Echo "Sorry, wrong selection";;
        Esac
        Echo -en "\n\n\t\t\thit any key to continue"
        Read -n 1 line
Done
Clear
```

Viết ứng dụng với Shell (tt)

Đọan script trên sẽ tạo ra menu có các chức năng như sau:

```
                    Sys Admin Menu

        1. Display disk space
        2. Display logged on users
        3. Display memory usage
        0. Exit program


              Enter option:
```

Hỏi & Đáp